Text and Audio Data

Minh Quân **Nhất Khôi**

Proofreading & Revision

Đoàn Lâm Phúc Khang **Trần Chí Công**

Nguyễn Lê Ngọc Khôi **Ngô Lam Thy**

I. Dẫn nhập

Trong thời đại trí tuệ nhân tạo, dữ liệu không chỉ tồn tại dưới dạng con số hay bảng tính mà còn xuất hiện dưới những dạng gần gũi hơn với con người như văn bản, giọng nói, bản ghi âm và nội dung số. Trong đó, dữ liệu văn bản giúp máy tính tiếp cận nội dung ngôn ngữ thông qua chữ viết, từ ngữ và ngữ cảnh; còn dữ liệu âm thanh mang thông tin dưới dạng tín hiệu theo thời gian, thể hiện giọng nói, nhịp điệu, cao độ và cảm xúc. Vì vậy, việc hiểu đặc điểm và quy trình xử lý dữ liệu văn bản, dữ liệu âm thanh là nền tảng quan trọng để xây dựng các ứng dụng AI như chatbot, nhận dạng giọng nói, chuyển giọng nói thành văn bản và phân tích cảm xúc.



Hình 1: Minh họa dữ liệu văn bản và dữ liệu âm thanh.

Mục lục

I. Dẫn nhập **1**

II. Dữ liệu văn bản và dữ liệu âm thanh **3**

II.1. Dữ liệu văn bản 3

II.2. Dữ liệu âm thanh 4

II.3. Ứng dụng của dữ liệu trí tuệ nhân tạo 4

III. Quy trình xử lý dữ liệu âm thanh và văn bản **5**

III.1. Một số các công cụ xử lý 5

III.2. Quy trình xử lý dữ liệu âm thanh 6

III.3. Quy trình xử lý dữ liệu văn bản 9

IV. Quiz **11**

V. Lời kết **13**

Tài liệu tham khảo **14**

II. Dữ liệu văn bản và dữ liệu âm thanh

II.1. Dữ liệu văn bản

Định nghĩa

Dữ liệu văn bản là loại dữ liệu được cấu thành từ các ký hiệu rời rạc, ví dụ như ký tự, từ, câu, đoạn văn hoặc toàn bộ tài liệu. Khác với dữ liệu số hay tín hiệu liên tục, văn bản thường mang tính phi cấu trúc vì không được tổ chức sẵn theo hàng, cột hay một khuôn mẫu cố định.

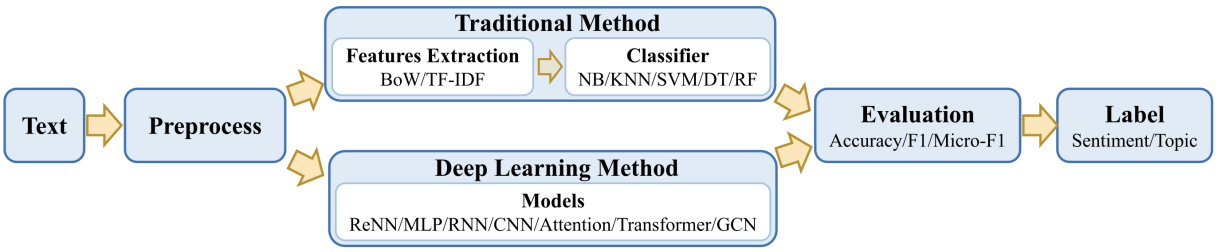
Để máy tính có thể xử lý văn bản, nội dung văn bản thường được chia thành các đơn vị nhỏ hơn gọi là token, có thể là từ, ký tự hoặc cụm từ ngắn như n-grams. Đây là bước quan trọng trong Xử lý Ngôn ngữ Tự nhiên, giúp máy tính khai thác thông tin từ những dữ liệu vốn được tạo ra bằng ngôn ngữ con người.

Cách biểu diễn dữ liệu văn bản

Dữ liệu văn bản thường được biểu diễn dưới dạng một chuỗi token theo thứ tự xuất hiện trong câu hoặc tài liệu. Sau đó, các token này được chuyển thành vector số hoặc ma trận embedding để mô hình có thể xử lý. Có thể hiểu đơn giản dữ liệu văn bản sau khi biểu diễn thường có dạng Tokens $\times$ Embedding Dimension.

Dữ liệu văn bản có đặc điểm nổi bật là mang tính biến đổi cao và thường mơ hồ về mặt ý nghĩa. Một từ có thể đảm nhận nhiều vai trò ngữ pháp khác nhau, các cụm từ đa từ (multi-word expressions) có thể mang nghĩa riêng, và một từ cũng có thể có nhiều nghĩa tùy theo ngữ cảnh. Vì vậy, khi xử lý văn bản, máy tính không chỉ nhận diện từng từ riêng lẻ mà còn cần phân tích mối quan hệ giữa các từ và bối cảnh xung quanh.

Vì văn bản ở dạng ký hiệu rời rạc nên máy tính không thể xử lý trực tiếp như con người. Do đó, văn bản cần được chuyển thành vector số thông qua các phương pháp như Bag-of-Words, TF-IDF, word embedding hoặc contextual embedding. Quá trình biểu diễn này giúp mô hình nắm bắt thông tin cú pháp, quan hệ ngữ nghĩa và ngữ cảnh của từ, từ đó làm cơ sở cho các bước xử lý văn bản tiếp theo.



Hình 2: Quy trình phân loại văn bản trích theo Li et al.

II.2. Dữ liệu âm thanh

Định nghĩa

Dữ liệu âm thanh là loại dữ liệu được tạo ra từ các tín hiệu âm thanh trong môi trường thực tế. Trong đó, tiếng nói là một dạng dữ liệu âm thanh quan trọng vì gắn trực tiếp với hoạt động giao tiếp, truyền tải thông tin, suy nghĩ và cảm xúc của con người.

Về mặt kỹ thuật, âm thanh tự nhiên ban đầu là tín hiệu liên tục ở dạng tương tự. Để máy tính có thể lưu trữ và xử lý, tín hiệu này cần được số hóa thông qua quá trình lấy mẫu và lượng tử hóa. Sau quá trình đó, âm thanh được biểu diễn thành chuỗi giá trị số theo thời gian, giúp máy tính có thể tiếp nhận và phân tích bằng các mô hình tính toán.

Cách biểu diễn dữ liệu âm thanh

Dữ liệu âm thanh sau khi số hóa thường được biểu diễn dưới dạng waveform, tức là một chuỗi các mẫu âm thanh theo thời gian. Nếu âm thanh có một kênh, dữ liệu có thể hiểu là Samples $\times$ 1; nếu là stereo, dữ liệu có thể có dạng Samples $\times$ 2. Ngoài waveform, âm thanh cũng thường được chuyển thành spectrogram để biểu diễn sự thay đổi của tần số theo thời gian, có thể hiểu đơn giản là Time Frames $\times$ Frequency Bins.

Dữ liệu âm thanh có tính biến thiên cao và khá phức tạp. Cùng một nội dung lời nói có thể khác nhau giữa từng người do giọng vùng miền, chất giọng, cao độ, cường độ và tốc độ nói. Ngoài ra, âm thanh trong thực tế còn dễ bị ảnh hưởng bởi tiếng ồn nền, âm vang, khoảng lặng hoặc chất lượng thiết bị ghi âm, khiến việc xử lý trở nên khó khăn hơn.

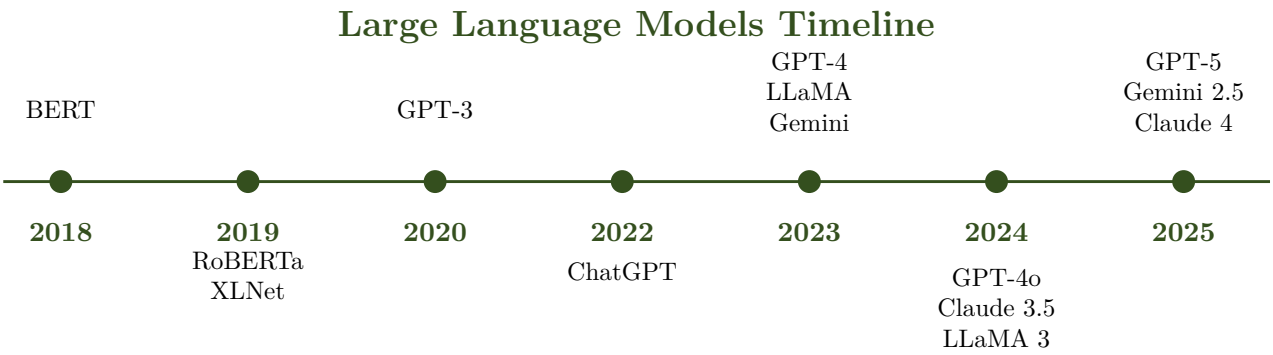
Vì âm thanh thô thường là chuỗi tín hiệu dài và dễ nhiễu, máy tính không thể hiểu trực tiếp ý nghĩa của nó nếu chưa qua xử lý. Do đó, dữ liệu âm thanh cần được tiền xử lý và trích xuất đặc trưng, chẳng hạn như loại bỏ nhiễu, cắt khoảng lặng, chuẩn hóa tín hiệu và chuyển waveform thành MFCCs hoặc spectrogram.

II.3. Ứng dụng của dữ liệu trí tuệ nhân tạo

Dữ liệu văn bản và dữ liệu âm thanh là hai nguồn dữ liệu quan trọng trong nhiều hệ thống trí tuệ nhân tạo hiện đại. Với dữ liệu văn bản, các mô hình AI có thể thực hiện nhiều tác vụ trong Xử lý Ngôn ngữ Tự nhiên như phân loại văn bản, phân tích cảm xúc, tóm tắt nội dung, dịch máy, hỏi đáp tự động và xây dựng chatbot.

Đối với dữ liệu âm thanh, đặc biệt là tiếng nói, AI thường được ứng dụng trong các bài toán như nhận dạng giọng nói, chuyển giọng nói thành văn bản, nhận diện người nói, phân loại âm thanh và phân tích cảm xúc qua giọng nói. Khác với văn bản, âm thanh còn chứa các thông tin phi ngôn ngữ như cao độ, cường độ, nhịp điệu và cảm xúc của người nói.

Trong thực tế, nhiều hệ thống AI hiện nay có xu hướng kết hợp cả dữ liệu văn bản và âm thanh để tạo ra trải nghiệm tự nhiên hơn. Khi kết hợp cả văn bản và âm thanh, mô hình có thể hiểu ngữ cảnh giao tiếp đầy đủ hơn, từ đó đưa ra kết quả chính xác và gần với cách con người tiếp nhận thông tin hơn.



Hình 3: Sự phát triển của các mô hình ngôn ngữ lớn.

III. Quy trình xử lý dữ liệu âm thanh và văn bản

III.1. Một số các công cụ xử lý

Trong Python, có nhiều thư viện hỗ trợ xử lý dữ liệu văn bản và dữ liệu âm thanh. Với dữ liệu văn bản, các công cụ như NLTK, Hugging Face Transformers và spaCy thường được dùng để tách từ, xử lý ngôn ngữ tự nhiên cơ bản, gán nhãn từ loại, nhận diện thực thể và sử dụng các mô hình ngôn ngữ hiện đại.



Hình 4: Một số công cụ xử lý dữ liệu văn bản.

Cụ thể, NLTK phù hợp với các thao tác nền tảng như tách câu, tách từ, loại bỏ stopwords hoặc phân tích ngữ pháp đơn giản. spaCy thường được dùng trong các quy trình xử lý văn bản thực tế nhờ tốc độ nhanh và khả năng nhận diện thực thể tốt. Trong khi đó, Hugging Face Transformers cung cấp nhiều mô hình hiện đại như BERT, GPT hoặc RoBERTa, giúp người dùng dễ dàng áp dụng các mô hình ngôn ngữ lớn vào những tác vụ xử lý văn bản khác nhau.

Với dữ liệu âm thanh, Librosa, SpeechBrain và Torchaudio là những thư viện tiêu biểu hỗ trợ đọc file âm thanh, phân tích tín hiệu, tạo waveform, spectrogram, trích xuất đặc trưng như MFCCs và xây dựng các mô hình xử lý tiếng nói. Bên cạnh đó, mỗi thư viện thường phù hợp với một nhu cầu khác nhau: Librosa mạnh về phân tích tín hiệu, Torchaudio thuận tiện khi kết hợp với PyTorch, còn SpeechBrain hỗ trợ nhiều bài toán xử lý tiếng nói như nhận dạng giọng nói và nhận dạng người nói.



Hình 5: Một số công cụ xử lý dữ liệu âm thanh.

III.2. Quy trình xử lý dữ liệu âm thanh

Quy trình xử lý dữ liệu âm thanh gồm 4 bước chính: tải dữ liệu, tiền xử lý tín hiệu, trích xuất đặc trưng và chuẩn bị đưa vào mô hình. Trong phần này, chúng ta sử dụng dataset `librispeech_asr_demo` từ HuggingFace để minh họa từng bước.

Bước 1 Tải dữ liệu

Dữ liệu âm thanh thường được lưu dưới dạng file `.wav` hoặc `.mp3`. Khi đọc vào, ta thu được hai thông tin quan trọng: waveform là chuỗi giá trị số biểu diễn tín hiệu âm thanh theo thời gian, và sample rate là số mẫu được lấy trong mỗi giây — sample rate càng cao thì âm thanh càng chi tiết. Ví dụ với sample rate 16000 Hz, mỗi giây âm thanh được biểu diễn bằng 16000 giá trị số.

```python
from datasets import load_dataset
# B1: Load file am thanh
dataset = load_dataset("hf-internal-testing/librispeech_asr_demo",
                       "clean", split="validation[:1]")
sample      = dataset[0]
waveform    = sample['audio']['array']
sample_rate = sample['audio']['sampling_rate']
print(f'Sample rate   : {sample_rate} Hz')
print(f'Waveform shape: {waveform.shape}')
print(f'Duration      : {len(waveform) / sample_rate:.2f} s')
print(f'Content       : {sample["text"]}')
```

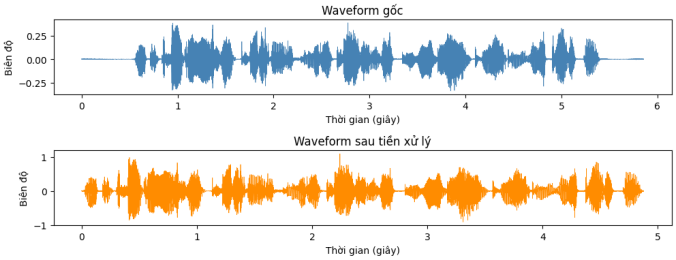
```
Sample rate   : 16000 Hz
Waveform shape: (93680,)
Duration      : 5.86 s
Content       : MISTER QUILTER IS THE APOSTLE OF THE MIDDLE CLASSES AND WE ARE GLAD TO
    WELCOME HIS GOSPEL
```

Code tải dataset từ HuggingFace, trích xuất waveform và sample rate từ mẫu đầu tiên, sau đó in ra các thông tin cơ bản để kiểm tra dữ liệu đã được đọc đúng chưa.

Bước 2 Tiền xử lý tín hiệu

Tín hiệu thô sau khi tải về thường chưa đồng đều và còn chứa nhiễu, do đó cần qua một số bước làm sạch trước khi phân tích. Chuẩn hóa biên độ đưa toàn bộ giá trị tín hiệu về khoảng $[-1, 1]$, giúp tránh ảnh hưởng của sự chênh lệch âm lượng giữa các file. Resample chuyển file về một sample rate thống nhất để đảm bảo tính nhất quán khi xử lý nhiều nguồn dữ liệu. Cắt khoảng lặng loại bỏ phần im lặng ở đầu và cuối file vốn không mang thông tin hữu ích.

```python
import numpy as np
import librosa
# Chuan hoa bien do
waveform_normalized = waveform / np.max(np.abs(waveform))
# Resample ve 22050 Hz
waveform_resampled = librosa.resample(waveform_normalized,orig_sr=sample_rate,target_sr=22050)
sample_rate_new = 22050
# Cat khoang lang
waveform_trimmed, _ = librosa.effects.trim(waveform_resampled, top_db=20)
print(f'Max before normalize: {np.max(np.abs(waveform)):.4f}')
print(f'Max after  normalize: {np.max(np.abs(waveform_normalized)):.4f}')
print(f'After resample - shape: {waveform_resampled.shape}, sr: {sample_rate_new} Hz')
print(f'After trim     - shape: {waveform_trimmed.shape}')
print(f'Duration after trim  : {len(waveform_trimmed) / sample_rate_new:.2f} s')
```




```
Max before normalize: 0.3879
Max after  normalize: 1.0000
After resample - shape: (129103,), sr: 22050 Hz
After trim     - shape: (107520,)
Duration after trim  : 4.88 s
```

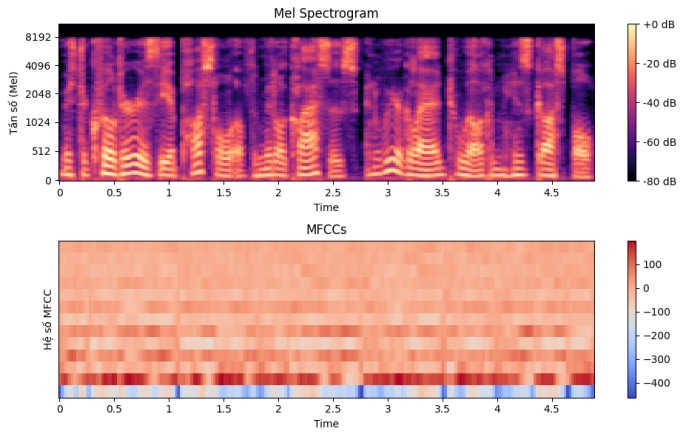
Hình 6: Kết quả của tiền xử lý

Code thực hiện lần lượt ba bước: chuẩn hóa biên độ bằng cách chia cho giá trị tuyệt đối lớn nhất, resample về 22050 Hz bằng librosa, và cắt khoảng lặng với ngưỡng 20 dB. Biểu đồ cho thấy rõ sự thay đổi biên độ và độ dài waveform trước và sau xử lý.

Bước 3 Trích xuất đặc trưng

Waveform thô là một chuỗi tín hiệu rất dài và dễ bị ảnh hưởng bởi nhiễu, mô hình không thể học hiệu quả trực tiếp từ đó. Vì vậy, cần chuyển waveform sang dạng biểu diễn giàu thông tin hơn. Mel Spectrogram biểu diễn sự thay đổi năng lượng theo tần số và thời gian, được nén theo thang Mel gần với cách tai người cảm nhận âm thanh. MFCCs (Mel-Frequency Cepstral Coefficients) trích xuất một tập hệ số nhỏ gọn từ Mel Spectrogram, giữ lại những đặc trưng quan trọng nhất của giọng nói và thường được dùng trong các bài toán nhận dạng giọng nói.

```python
# Mel Spectrogram,MFCCs
mel_spec = librosa.feature.melspectrogram(y=waveform_trimmed,sr=sample_rate_new,n_mels=128)
mel_spec_db = librosa.power_to_db(mel_spec, ref=np.max)
mfccs = librosa.feature.mfcc(y=waveform_trimmed,sr=sample_rate_new,n_mfcc=13)
print(f'Mel Spectrogram shape: {mel_spec_db.shape}')
print(f'  -> (Frequency Bins x Time Frames)')
print(f'MFCCs shape          : {mfccs.shape}')
print(f'  -> (n_mfcc x Time Frames)')
```




```
Mel Spectrogram shape: (128, 211)
  -> (Frequency Bins x Time Frames)
MFCCs shape          : (13, 211)
  -> (n_mfcc x Time Frames)
```

Hình 7: Kết quả của trích xuất đặc trưng

Code tính Mel Spectrogram với 128 dải tần số rồi chuyển sang đơn vị dB, tiếp theo tính MFCCs với 13 hệ số. Biểu đồ phía trên cho thấy phân bố năng lượng theo tần số và thời gian, biểu đồ phía dưới thể hiện 13 hệ số MFCC biến thiên theo thời gian.

Bước 4 Chuẩn bị đưa vào mô hình

Sau khi trích xuất đặc trưng, dữ liệu cần trải qua một bước chuẩn hóa cuối trước khi đưa vào mô hình học sâu. Chuẩn hóa về mean = 0 và std = 1 giúp các đặc trưng có cùng scale, tránh tình trạng một số hệ số có giá trị quá lớn làm ảnh hưởng đến quá trình huấn luyện. Cuối cùng, dữ liệu được chuyển sang tensor PyTorch với shape `(batch, n_mfcc, time_frames)`.

```python
import torch
# Chuan hoa MFCCs
mfccs_normalized = (mfccs - mfccs.mean()) / mfccs.std()
# Chuyen sang tensor
tensor = torch.tensor(mfccs_normalized, dtype=torch.float32).unsqueeze(0)
print(f'Mean before normalize: {mfccs.mean():.4f}')
print(f'Std  before normalize: {mfccs.std():.4f}')
print(f'Mean after  normalize: {mfccs_normalized.mean():.4f}')
print(f'Std  after  normalize: {mfccs_normalized.std():.4f}')
print(f'Tensor shape: {tensor.shape}')
print(f'  -> (batch x n_mfcc x Time Frames)')
print(f'Dtype: {tensor.dtype}')
```

```
Mean before normalize: -2.3877
Std  before normalize: 73.0981
Mean after  normalize: -0.0000
Std  after  normalize: 1.0000
Tensor shape: torch.Size([1, 13, 211])
  -> (batch x n_mfcc x Time Frames)
Dtype: torch.float32
```

Code chuẩn hóa MFCCs bằng cách trừ mean và chia std, sau đó thêm chiều batch bằng `unsqueeze(0)` và chuyển sang tensor `float32`. Output cho thấy mean và std trước/sau chuẩn hóa để xác nhận kết quả đúng.

III.3. Quy trình xử lý dữ liệu văn bản

Quy trình xử lý dữ liệu văn bản theo hướng Transformer gồm 3 bước chính: làm sạch văn bản, tokenization bằng BERT và chuẩn bị đầu vào cho mô hình. Trong phần này, chúng ta sử dụng một số câu văn tiếng Anh đơn giản để minh họa từng bước.

Bước 1 Làm sạch văn bản

Văn bản thô thường chứa nhiều thành phần không cần thiết như ký tự đặc biệt, dấu câu hay khoảng trắng thừa. Bước làm sạch giúp chuẩn hóa văn bản về một dạng thống nhất trước khi đưa vào tokenizer. Các thao tác phổ biến gồm chuyển toàn bộ chữ về chữ thường và loại bỏ ký tự đặc biệt.

```python
import re

texts = [
    "Hello! This is an example sentence for text processing.",
    "Natural Language Processing helps computers understand human language."
]

def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

cleaned = [clean_text(t) for t in texts]
for i, t in enumerate(cleaned):
    print(f'Text {i+1}: {t}')
```

```
Text 1: hello this is an example sentence for text processing
Text 2: natural language processing helps computers understand human language
```

Code chuyển toàn bộ văn bản về chữ thường bằng `lower()`, sau đó dùng regex để loại bỏ mọi ký tự không phải chữ cái hoặc khoảng trắng, và chuẩn hóa khoảng trắng thừa. Kết quả là các câu sạch, thống nhất và sẵn sàng đưa vào tokenizer.

Bước 2 Tokenization và Encoding bằng BERT

Khác với tokenization thủ công, BERT Tokenizer tự động xử lý toàn bộ quá trình: tách từ thành token, chuyển token thành ID số nguyên, thêm các token đặc biệt `[CLS]` ở đầu và `[SEP]` ở cuối câu. Lưu ý rằng với BERT, không nên loại bỏ stopwords thủ công vì các từ như “is”, “an”, “the” vẫn đóng vai trò quan trọng trong việc xác định ngữ cảnh của câu.

```python
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

encoded = tokenizer(
    cleaned,
```

```
    padding=True,
    truncation=True,
    max_length=20,
    return_tensors='pt'
)

print('Input IDs:')
print(encoded['input_ids'])
print(f'Shape: {encoded["input_ids"].shape}')
print(f'  -> (num_sentences x max_length)')
```

```
Input IDs:
tensor([[  101,  7592,  2023,  2003,  2019,  2742,  6251,  2005,  3793,  6364,
           102,     0,     0,     0,     0,     0,     0,     0,     0,     0],
        [  101,  3019,  2653,  6364,  7126,  7588,  3305,  2529,  2653,  1012,
           102,     0,     0,     0,     0,     0,     0,     0,     0,     0]])
Shape: torch.Size([2, 20])
  -> (num_sentences x max_length)
```

Code truyền thẳng danh sách `cleaned` vào BERT Tokenizer. Token `[CLS]` (ID 101) được thêm vào đầu và `[SEP]` (ID 102) được thêm vào cuối mỗi câu theo chuẩn BERT. Shape `(2, 20)` cho thấy có 2 câu, mỗi câu được biểu diễn bằng 20 ID.

Bước 3 Padding và Attention mask

Các câu trong cùng một batch thường có độ dài khác nhau. Để đưa vào mô hình, ta cần chúng có cùng độ dài bằng cách thêm token `[PAD]` (ID 0) vào cuối câu ngắn hơn. Đồng thời, attention mask được tạo ra để cho mô hình biết vị trí nào là token thật (giá trị 1) và vị trí nào là padding (giá trị 0).

```
print('Attention Mask:')
print(encoded['attention_mask'])
print()
for i in range(len(cleaned)):
    n_real = encoded['attention_mask'][i].sum().item()
    n_pad  = encoded['attention_mask'].shape[1] - n_real
    print(f'Text {i+1}: {int(n_real)} real tokens, {int(n_pad)} padding tokens')
```

```
Attention Mask:
tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]])

Text 1: 11 real tokens, 9 padding tokens
Text 2: 10 real tokens, 10 padding tokens
```

Attention mask cho thấy rõ vị trí token thật và padding trong mỗi câu. Mô hình sẽ dựa vào attention mask để bỏ qua các vị trí padding trong quá trình tính toán, đảm bảo padding không ảnh hưởng đến kết quả học.

IV. Quiz

1. Dữ liệu văn bản khác với dữ liệu số ở điểm nào?
   (*a*) Dữ liệu văn bản luôn có cấu trúc rõ ràng theo hàng và cột.
   (*b*) Dữ liệu văn bản được cấu thành từ các ký hiệu rời rạc và thường phi cấu trúc.
   (*c*) Dữ liệu văn bản chỉ gồm các con số.
   (*d*) Dữ liệu văn bản không thể đưa vào mô hình AI.

2. Token trong xử lý văn bản là gì?
   (*a*) Một loại mô hình ngôn ngữ lớn.
   (*b*) Đơn vị nhỏ nhất của tín hiệu âm thanh.
   (*c*) Đơn vị nhỏ hơn mà văn bản được chia thành, ví dụ từ hoặc ký tự.
   (*d*) Một vector số dùng để biểu diễn ảnh.

3. Dữ liệu âm thanh sau khi số hóa thường được biểu diễn dưới dạng nào?
   (*a*) Ma trận ảnh RGB.
   (*b*) Waveform, chuỗi các mẫu âm thanh theo thời gian.
   (*c*) Danh sách các từ.
   (*d*) Tensor 4 chiều.

4. Sample rate trong dữ liệu âm thanh cho biết điều gì?
   (*a*) Số từ trong một câu văn bản.
   (*b*) Số pixel trong một ảnh.
   (*c*) Số mẫu âm thanh được lấy trong một giây.
   (*d*) Số kênh màu của ảnh.

5. Vì sao cần chuẩn hóa biên độ waveform trước khi xử lý?
   (*a*) Để tăng độ dài của tín hiệu âm thanh.
   (*b*) Để chuyển waveform thành văn bản.
   (*c*) Để tránh ảnh hưởng của sự chênh lệch âm lượng giữa các file khác nhau.
   (*d*) Để tăng sample rate lên 22050 Hz.

6. MFCCs trong xử lý âm thanh dùng để làm gì?
   (*a*) Chuyển văn bản thành giọng nói.
   (*b*) Trích xuất hệ số đặc trưng từ Mel Spectrogram, giữ lại thông tin quan trọng của giọng nói.
   (*c*) Tăng tốc độ xử lý video.
   (*d*) Mã hóa văn bản thành ID số nguyên.

7. Trong xử lý văn bản, stopwords là gì?

(*a*) Những từ quan trọng nhất trong câu.

(*b*) Những từ phổ biến ít mang nghĩa thường bị loại bỏ trước khi xử lý.

(*c*) Những từ được mã hóa thành ID số.

(*d*) Những từ chỉ xuất hiện một lần trong toàn bộ văn bản.

8. Đoạn code sau thực hiện bước nào trong quy trình xử lý văn bản?

```
text = text.lower()
text = re.sub(r''[^a-z\s]'', '', text)
```

(*a*) Encoding, chuyển token thành ID số.

(*b*) Padding, thêm token đặc biệt vào cuối câu.

(*c*) Làm sạch văn bản, chuyển chữ thường và loại bỏ ký tự đặc biệt.

(*d*) Tokenization, tách câu thành danh sách từ.

9. Attention mask trong BERT có vai trò gì?

(*a*) Tăng số lượng token trong câu.

(*b*) Cho mô hình biết vị trí nào là token thật và vị trí nào là padding.

(*c*) Chuyển token thành vector embedding.

(*d*) Loại bỏ stopwords khỏi câu.

10. Shape `torch.Size([1, 13, 211])` sau khi xử lý MFCCs có ý nghĩa gì?

(*a*) 1 kênh màu, 13 pixel chiều cao, 211 pixel chiều rộng.

(*b*) 1 batch, 13 hệ số MFCC, 211 khung thời gian.

(*c*) 1 câu văn bản, 13 từ, 211 ký tự.

(*d*) 1 file âm thanh, 13 giây, 211 mẫu.

Lời giải

1. B. Dữ liệu văn bản được cấu thành từ ký hiệu rời rạc và thường không có cấu trúc cố định.
2. C. Token là đơn vị nhỏ hơn mà văn bản được chia thành, có thể là từ, ký tự hoặc cụm từ ngắn.
3. B. Waveform là chuỗi giá trị số biểu diễn tín hiệu âm thanh theo thời gian.
4. C. Sample rate cho biết số mẫu âm thanh được lấy trong một giây càng cao thì âm thanh càng chi tiết.
5. C. Chuẩn hóa biên độ giúp tránh sự chênh lệch âm lượng giữa các file, đảm bảo tín hiệu đồng đều.
6. B. MFCCs trích xuất các hệ số đặc trưng nhỏ gọn từ Mel Spectrogram, thường dùng trong nhận dạng giọng nói.
7. B. Stopwords là những từ phổ biến như "is", "the", "an" ít mang nghĩa và thường bị loại bỏ.
8. C. Đoạn code chuyển văn bản về chữ thường và loại bỏ ký tự đặc biệt đây là bước làm sạch văn bản.
9. B. Attention mask giúp mô hình phân biệt token thật (giá trị 1) và token padding (giá trị 0).
10. B. Shape `[1, 13, 211]` tương ứng với 1 batch, 13 hệ số MFCC và 211 khung thời gian.

V. Lời kết

Dữ liệu văn bản và dữ liệu âm thanh là hai dạng dữ liệu quan trọng trong các hệ thống trí tuệ nhân tạo hiện đại, vì chúng gắn trực tiếp với cách con người giao tiếp và truyền tải thông tin. Văn bản giúp máy tính tiếp cận nội dung ngôn ngữ thông qua ký tự, từ và ngữ cảnh, trong khi âm thanh bổ sung thêm các yếu tố như giọng nói, cao độ, cường độ, nhịp điệu và cảm xúc. Tuy nhiên, cả hai loại dữ liệu này đều không thể được mô hình xử lý trực tiếp nếu chưa trải qua các bước tiền xử lý và biểu diễn phù hợp.

Thông qua việc tìm hiểu đặc điểm, cách biểu diễn và quy trình xử lý dữ liệu văn bản và âm thanh, có thể thấy rằng AI không chỉ đơn thuần làm việc với con số, mà còn đang ngày càng tiến gần hơn đến khả năng hiểu ngôn ngữ và giao tiếp tự nhiên của con người. Đây là nền tảng quan trọng cho nhiều ứng dụng thực tế như chatbot, nhận dạng giọng nói, chuyển giọng nói thành văn bản, phân tích cảm xúc và các hệ thống AI đa phương thức.

Tài liệu tham khảo

1. Siebers, P., Janiesch, C., & Zschech, P. (2022). A survey of text representation methods and their genealogy. *IEEE Access, 10*, 96492–96513. https://doi.org/10.1109/ACCESS.2022.3205719

2. Kheddar, H., Hemis, M., & Himeur, Y. (2024). Automatic speech recognition using advanced deep learning approaches: A survey. *Information Fusion, 109*, 102422. https://doi.org/10.1016/j.inffus.2024.102422

- Hết -